

DOCUMENT 3: COMPLETE GUIDE TO EVERY MACHINE LEARNING MODEL USED IN THIS PROJECT

Simple English Guide: HOW, WHY, and WHEN for All 6 Regression Algorithms

Introduction

This document provides an in-depth, plain-English breakdown of all **6 Machine Learning Regression Algorithms** developed, tuned, and benchmarked in our **Loan Amount Prediction System**.

For every model, this document explains:

1. **WHAT it is:** Plain-English intuition and real-world analogy.
 2. **HOW it works:** Mathematical principles explained simply.
 3. **WHY we chose it:** Its purpose in our loan underwriting project.
 4. **WHEN to use it (and when NOT to):** Production guidance.
 5. **Exact Performance in our Project:** MAE, MSE, RMSE, R^2 , and ranking analysis.
-

Model 1: Linear Regression

1. WHAT is Linear Regression?

Linear Regression is the grandfather of predictive modeling. Imagine drawing a single straight line through a scatter plot of dots so that the line sits as close to every dot as possible. It assumes that as input numbers go up or down, the output moves in a constant, straight-line proportion.

2. HOW does it work?

- **Mathematical Equation:**

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

Where:

- $\hat{y}$ is the predicted loan amount.
- β_0 is the intercept (base loan amount).
- $X_1, X_2, \dots$ are the applicant features (income, assets, requested loan).

- $\beta_1, \beta_2, \dots$ are the learned weights (slopes) showing how much the loan increases for each unit change in that feature.
- **Optimization (Ordinary Least Squares):** The algorithm calculates the mathematical derivative to find the exact weights that minimize the sum of squared errors:

$$\text{Loss} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

3. WHY we chose it:

It provides our **foundational benchmark**. Before testing complex algorithms, every data scientist must test Linear Regression to establish a performance baseline. It is also 100% transparent and interpretable.

4. WHEN to use it:

- **USE WHEN:** The relationship between features and target is mostly linear, interpretability is required by law, and you need lightning-fast inference.
- **DO NOT USE WHEN:** The data has sharp threshold cutoffs (e.g., credit score cliff-effects) or non-linear compounding interactions.

5. Performance in our Loan Project:

- **MAE:** █3,04,394.08
- **MSE:** 2.08×10^{11}
- **RMSE:** █4,56,024.85
- **Test R^2 Score:** 0.9847 (Train R^2 : 0.9864)
- **Rank:** #5 out of 6
- **Analysis:** Linear Regression performed remarkably well because loan amount requested and asset values have strong linear correlations with sanctioned amounts, but it underperformed compared to non-linear tree ensembles.

Model 2: Polynomial Regression (Degree 2 with Ridge Regularization)

1. WHAT is Polynomial Regression?

If Linear Regression is a straight ruler, Polynomial Regression is a flexible curve. It creates curved shapes by taking original features and multiplying them together or squaring them (e.g., Income^2 or $\text{Income} \times \text{AssetValue}$).

2. HOW does it work?

- **Feature Transformation (`PolynomialFeatures(degree=2)`):** It automatically generates all quadratic combinations:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 (X_1 X_2)$$

- **Ridge Regularization (*L2 Penalty*):** Squaring features creates multicollinearity (features that heavily overlap), which can cause mathematical instability. To prevent this, we add a Ridge penalty:

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \sum \beta_j^2 \quad (\alpha = 100.0)$$

This penalizes oversized weights and ensures a smooth, stable curve.

3. WHY we chose it:

In banking, financial limits rarely scale in a perfectly straight line. For example, high-income applicants with high collateral experience a compounding boost in loan eligibility. Polynomial interactions capture these multi-variable synergies.

4. WHEN to use it:

- **USE WHEN:** Data has smooth curvatures or interaction effects between pairs of features.
- **DO NOT USE WHEN:** You have hundreds of features (which causes feature explosion) or higher degrees (> 3), which causes wild overfitting.

5. Performance in our Loan Project:

- **MAE:** ■2,81,828.43
- **MSE:** 1.85×10^{11}
- **RMSE:** ■4,30,132.33
- **Test R^2 Score:** 0.9864 (Train R^2 : 0.9884)
- **Rank:** #3 out of 6
- **Analysis:** By capturing interaction curves, it reduced the RMSE by over ■25,000 compared to standard Linear Regression.

Model 3: Decision Tree Regression

1. WHAT is a Decision Tree?

A Decision Tree is like playing a game of "20 Questions." It makes predictions by asking a sequence of Yes/No questions about the applicant (e.g., "Is Credit Score > 750 ? If Yes, is Income $> \text{■}2$ Lakhs?").

2. HOW does it work?

- **Recursive Splitting:** At every step (node), the algorithm searches through every feature and every possible numerical cutoff to find the split that maximizes the reduction in Mean Squared Error (variance reduction):

$$\Delta \text{MSE} = \text{MSE}_{\text{parent}} - \left(\frac{N_{\text{left}}}{N} \text{MSE}_{\text{left}} + \frac{N_{\text{right}}}{N} \text{MSE}_{\text{right}} \right)$$

- **Leaf Predictions:** Once an applicant reaches a terminal leaf node, the predicted loan amount is simply the average loan amount of all training applicants who landed in that same bucket.

- **Pruning & Constraints:** To prevent the tree from growing until each leaf contains only 1 applicant (pure memorization), we constrained it with `max_depth=9` and `min_samples_leaf=10`.

3. WHY we chose it:

Banking policies operate using strict rule-based tiers (e.g., "Applicants with credit score below 600 get an automatic haircut"). Decision trees mirror how human loan officers think.

4. WHEN to use it:

- **USE WHEN:** You need human-readable decision rules and the data has categorical step changes.
- **DO NOT USE WHEN:** You need smooth numeric predictions. Single decision trees produce jagged "staircase" predictions and have high variance.

5. Performance in our Loan Project:

- **MAE:** ■274,960.44
 - **MSE:** 1.89×10^{11}
 - **RMSE:** ■434,535.01
 - **Test R^2 Score:** 0.9861 (Train R^2 : 0.9900)
 - **Rank:** #4 out of 6
 - **Analysis:** Captures segmented credit tiers effectively, but its step-wise approximations result in slightly higher RMSE than multi-tree ensembles.
-

Model 4: Random Forest Regression

1. WHAT is a Random Forest?

"Wisdom of the crowd." Instead of relying on a single decision tree that might make mistakes, Random Forest grows **100 independent decision trees** and averages their predictions together.

2. HOW does it work?

- **Bootstrap Aggregation (Bagging):** Each tree is trained on a random sample of the data chosen with replacement (some rows are duplicated, some left out).
- **Feature Subsampling:** At every split, each tree only gets to consider a random subset of features. This prevents all 100 trees from looking identical.
- **Averaging:**

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

The individual errors of individual trees cancel each other out, producing a smooth, robust prediction.

3. WHY we chose it:

Random Forest is one of the most reliable "off-the-shelf" algorithms in machine learning. It almost never overfits severely and handles non-linearities and outliers with ease.

4. WHEN to use it:

- **USE WHEN:** You want high accuracy with minimal tuning, and your dataset contains both numerical and categorical features with complex relationships.
- **DO NOT USE WHEN:** You need very small model file sizes (a forest of 100 deep trees can take 10+ megabytes).

5. Performance in our Loan Project:

- **MAE:** ■258,950.02
 - **MSE:** 1.60×10^{11}
 - **RMSE:** ■399,623.06
 - **Test R^2 Score:** 0.9883 (Train R^2 : 0.9938)
 - **Rank:** #2 out of 6
 - **Analysis:** Dramatically reduced error compared to a single Decision Tree (RMSE dropped from ■4.34L to ■3.99L), demonstrating the power of ensemble averaging.
-

Model 5: Gradient Boosting Regression (THE CHAMPION MODEL)

1. WHAT is Gradient Boosting?

If Random Forest is a committee voting simultaneously, Gradient Boosting is a master apprentice system. It builds trees **one after another in sequence**. Each new tree focuses specifically on the mistakes (residuals) made by the previous trees!

2. HOW does it work?

1. Initial Baseline: The model starts by predicting the average loan amount for everyone: $F_0(x) = \bar{y}$.

2. Compute Residual Errors: For every applicant, calculate the error:

$$r_{\{i, m\}} = y_i - F_{\{m-1\}}(x_i)$$

3. Fit a New Shallow Tree: A small tree ($max_depth=5$) is trained to predict these residual errors.

4. Update with Shrinkage (Learning Rate η):

$$F_m(x) = F_{\{m-1\}}(x) + \eta \cdot T_m(x) \quad (\eta = 0.08)$$

Multiplying by a small learning rate ensures the model takes small, careful steps and avoids overshooting the target.

5. Repeat 150 Times: Over 150 rounds, the residual errors shrink closer and closer to zero.

3. WHY we chose it:

Gradient Boosting is the undisputed gold standard for tabular data in machine learning competitions (Kaggle) and enterprise credit scoring. It captures intricate non-linearities better than any other classical algorithm.

4. WHEN to use it:

- **USE WHEN:** Maximizing predictive accuracy on tabular data is your top priority.
- **DO NOT USE WHEN:** Training time is severely limited or data is extremely noisy with severe mislabeled outliers.

5. Performance in our Loan Project:

- **MAE:** ■250,815.84 (Lowest error!)
 - **MSE:** 1.53×10^{11}
 - **RMSE:** ■391,191.33 (Lowest RMSE across all models!)
 - **Test R^2 Score:** 0.9888 (Highest R^2 score!)
 - **Train R^2 :** 0.9948
 - **Rank:** #1 UNDISPUTED CHAMPION
 - **Analysis:** Gradient Boosting struck the perfect balance between bias and variance, achieving the highest precision on the test set.
-

Model 6: Support Vector Regression (SVR with RBF Kernel)

1. WHAT is Support Vector Regression?

Imagine building a "paved highway" (a margin of width ϵ) along the data. SVR tries to fit as many data points as possible inside this highway. As long as points fall inside the highway, they cause zero penalty. Only points that spill outside the road create an error.

2. HOW does it work?

- **ϵ -Insensitive Loss Function:**

$$L_{\epsilon}(y, \hat{y}) = \begin{cases} 0 & \text{if } |y - \hat{y}| \leq \epsilon \\ |y - \hat{y}| - \epsilon & \text{otherwise} \end{cases}$$

- **Kernel Trick (Radial Basis Function - RBF):** The RBF kernel maps the data into an infinite-dimensional mathematical space where non-linear patterns become linearly separable:

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

- **Target Scaling (TransformedTargetRegressor):** Because SVR calculates geometric distances, large targets (millions of Rupees) can cause numerical instability. We wrapped SVR in a target scaler so the optimizer works smoothly.

3. WHY we chose it:

SVR offers strong theoretical guarantees against overfitting and provides a completely different mathematical approach (convex geometric optimization) compared to tree-based methods.

4. WHEN to use it:

- **USE WHEN:** The dataset has high dimensionality or smooth non-linear manifolds, and you need robust resistance to small fluctuations within margin *lepsilon*.
- **DO NOT USE WHEN:** The dataset has hundreds of thousands of rows (SVR scales with $O(N^2)$ to $O(N^3)$ computational complexity).

5. Performance in our Loan Project:

- **MAE:** █332,929.53
- **MSE:** 2.86×10^{11}
- **RMSE:** █534,882.84
- **Test R^2 Score:** 0.9790 (Train R^2 : 0.9967)
- **Rank:** #6 out of 6
- **Analysis:** SVR performed with high accuracy ($R^2 = 0.979$), but the large scale and tabular boundaries of banking data favored tree-based ensembles.

Summary: Final Algorithm Comparison Matrix

Rank	Algorithm	MAE (█)	RMSE (█)	Test R^2	Train R^2	Best Use-Case
#1	Gradient Boosting Regression	█2.50 Lakh	█3.91 Lakh	0.9888	0.9948	Production Deployment
#2	Random Forest Regression	█2.58 Lakh	█3.99 Lakh	0.9883	0.9938	High Reliability & Stability
#3	Polynomial Regression (Deg-2)	█2.81 Lakh	█4.30 Lakh	0.9864	0.9884	Smooth Feature Interactions
#4	Decision Tree Regression	█2.74 Lakh	█4.34 Lakh	0.9861	0.9900	Transparent Policy Rules
#5	Linear Regression	█3.04 Lakh	█4.56 Lakh	0.9847	0.9864	Fast, Interpretable Baseline
#6	Support Vector Regression	█3.32 Lakh	█5.34 Lakh	0.9790	0.9967	Margin-Based Boundaries